



ДИСЦИПЛИНА	<u>Разработка приложений на языке Котлин</u> (полное наименование дисциплины без сокращений)
ИНСТИТУТ	<u>информационных технологий</u>
КАФЕДРА	<u>информационных технологий в атомной энергетике</u> полное наименование кафедры
ВИД УЧЕБНОГО МАТЕРИАЛА	<u>Лекция</u> (в соответствии с пп 1-11)
ПРЕПОДАВАТЕЛЬ	<u>Золотухин Святослав Александрович</u> (фамилия, имя, отчество)
СЕМЕСТР	<u>5 семестр 2025 – 2026 учебный год</u> (указать семестр обучения, учебный год)



Лекция 3. Разработка приложений на языке Котлин

ФИО преподавателя: Золотухин Святослав Александрович

e-mail: zolotuhin@mirea.ru



ООП. Классы и объекты

Для определения класса применяется ключевое слово `class`:

```
class Person
```

```
class Person { } // можно так
```



ООП. Классы и объекты

Класс представляет новый тип данных, поэтому можно определять переменные этого типа, например:

```
fun main() {
```

```
    val kate: Person
```

```
    val john: Person
```

```
    val andrew: Person
```

```
}
```

Функция main в Kotlin не помещается в отдельных класс, а всегда определяется вне какого-либо класса.

```
class Person
```



ООП. Создание объекта класса

Для создания объекта класса необходимо вызвать конструктор данного класса. По умолчанию для класса компилятор генерирует пустой конструктор, который можно использовать:

```
val tom: Person = Person()
```

Например, создадим три объекта класса Person:

```
fun main() {  
    val kate: Person = Person()  
    val john: Person = Person()  
    val andrew: Person = Person()  
}  
  
class Person
```



ООП. Свойства класса

Каждый класс может хранить некоторые данные или состояние в виде свойств. **Свойства** представляют **переменные**, определенные на уровне класса с ключевыми словами **val** и **var** (**неизменяемые/изменяемые**)

Свойства обязательно должны иметь начальное значение.

```
class Person{  
    var name: String = "Без имени"  
    var age: Int = 18  
}
```



ООП. Свойства класса

Поскольку эти свойства определены с **var**, то можно изменить их начальные значения:

```
fun main() {  
    val andrew: Person = Person() // создаем объект  
    println(andrew.name)    // Без имени  
    println(andrew.age)    // 18  
    andrew.name = "Андрей"  
    andrew.age = 25  
    println(andrew.name)    // Андрей  
    println(andrew.age)    // 25  
}  
  
class Person{  
    var name: String = "Без имени"  
    var age: Int = 18  
}
```



ООП. Свойства класса

Обращение к свойствам происходит по имени переменной, которая представляет объект, после точки указывается имя свойства

```
val personName : String = andrew.name
```

Как видно из предыдущего слайда, установка значения свойства происходит таким образом:

```
andrew.name = "Андрей"
```




ООП. Функции класса

Класс также может содержать функции (методы). Методы определяют поведение объектов данного класса. В Котлин такие методы еще называют **member functions** или **функции-члены класса**.

```
class Person{  
    var name: String = "Без имени"  
    var age: Int = 18  
    fun sayHello(){  
        println("Привет, меня зовут $name")  
    }  
    fun personToString() : String{  
        return "Имя: $name  Возраст: $age"  
    }  
}
```

```
fun main() {  
    val peter = Person()  
    peter.name = "Петр"  
    peter.age = 27  
  
    peter.sayHello()  
    println(peter.personToString())  
}  
// Привет, меня зовут Петр  
Имя: Петр  Возраст: 27
```



ООП. Конструкторы

Для создания объекта необходимо вызвать конструктор класса. По умолчанию компилятор создает конструктор, который не принимает параметров и который можно использовать. Но также можно определять собственные конструкторы.

Для определения конструкторов применяется ключевое слово **constructor**.



ООП. Конструкторы

Классы в Kotlin могут иметь один первичный конструктор (**primary constructor**) и один или несколько вторичных конструкторов (**secondary constructor**).

Первичный конструктор является частью заголовка класса и определяется сразу после имени класса:

```
class Person constructor(_name: String){  
}
```

Если первичный конструктор не имеет никаких аннотаций или модификаторов доступа, как в данном случае, то ключевое слово `constructor` можно опустить



ООП. Инициализатор

Полученные через конструктор данные можно использовать для инициализации свойств класса. Для этого применяется блок **init**

```
class Person(_name: String){  
    val name: String  
    init{  
        name = _name  
    }  
}
```



ООП. Инициализатор

Цель инициализатора состоит в инициализации объекта при его создании. Стоит отметить, что в классе может быть определено одновременно несколько блоков инициализатора.

В данном случае в инициализаторе нет смысла, так как параметры первичного конструктора можно напрямую передавать свойствам.

```
class Person(_name: String){  
    val name: String = _name  
}
```



ООП. Первичный конструктор

Первичный конструктор также может использоваться для определения свойств:

```
fun main() {  
    val peter: Person = Person("Peter", 25)  
    println("Имя: ${peter.name} Возраст: ${peter.age}")  
}  
  
class Person(val name: String, var age: Int){  
}
```

Свойства определяются как и параметры, при этом их определение начинается с ключевого слова **val** (если их не планируется изменять) и **var** (если планируется изменять). В этом случае уже необязательно определять эти свойства в теле класса явно, так как их уже определяет конструктор **Person("Peter", 25)**



ООП. Вторичные конструкторы

Вторичные конструкторы применяются, чтобы определить доп параметры.

Если для класса определен первичный конструктор, то вторичный конструктор должен вызывать первичный с помощью ключевого слова `this`

```
class Person(_name: String){  
    val name: String = _name  
    var age: Int = 0  
  
    constructor(_name: String, _age: Int) : this(_name){  
        age = _age  
    }  
}
```



ООП. Пакеты и импорт

В Kotlin, пакеты представляют собой организационные блоки, которые группируют связанный функционал, такой как классы и функции, используемые для решения схожих задач. Это позволяет упорядочить и структурировать код. Внутри пакетов можно разместить классы и функции, которые служат для решения определенной задачи, и организовать их логически.

Для определения пакета применяется ключевое слово `package`, после которого идет имя пакета:

`package example`

Определение пакета помещается в самое начало файла. И все содержимое файла рассматривается как содержимое этого пакета.



ООП. Пакеты и импорт

Например, добавим в проект новый файл `example.kt`:

// Пакеты в Kotlin

// И определим в нем следующий код:

```
class Item(val name: String) {}  
  
fun display(item: Item) {  
    println("Item: ${item.name}")  
}
```

Чтобы использовать функционал этого пакета в другом файле необходимо применить директиву `import`. Здесь возможны различные способы подключения функционала из пакета. Можно подключить в целом весь пакет **`import example.*`** или по отдельности: **`import example.display`**

```
import example.Item
```



ООП. Пакеты и импорт. Псевдонимы

С помощью оператора `as` можно определять псевдоним для подключаемого типа и затем обращаться к этому типу через его псевдоним:

```
import example.display as displayItem
```

```
import example.Item as MyItem
```

```
fun main() {
```

```
    val myItem = MyItem("Sample Item")
```

```
    displayItem(myItem)
```

```
}
```

Здесь для функции `display()` определен псевдоним `displayItem`. И далее для обращения к этой функции надо использовать ее псевдоним **`displayItem(myItem)`**, также с **`MyItem`**



ООП. Модификаторы видимости

В Kotlin, каждый тип данных и его компоненты, такие как классы, объекты, интерфейсы, конструкторы, функции и свойства, имеют уровень видимости, который определяется модификатором доступа. Модификатор доступа определяет, где и как можно использовать эти типы и их компоненты. В Kotlin существуют следующие модификаторы доступа:

- **private**
- **protected**
- **internal**
- **public**



ООП. Модификаторы видимости

private: С типами и компонентами, помеченными этим модификатором, можно работать только внутри файла, в котором они определены. Компоненты класса с таким модификатором видны только внутри самого класса.

protected: Компоненты класса с этим модификатором доступны внутри класса, где они определены, а также в классах-наследниках.



ООП. Модификаторы видимости

internal: С типами и компонентами, помеченными этим модификатором, можно работать в любой части модуля, в котором они определены. Модуль представляет собой набор файлов Kotlin, скомпилированных вместе в одну структурную единицу. Это может быть, например, модуль в IntelliJ IDEA или проект Maven.

public: Типы и компоненты с этим модификатором видны в любой части программы. Однако если они определены в другом пакете, их все равно нужно явно импортировать.



ООП. Модификаторы видимости

Для установки уровня видимости модификатор ставится перед ключевыми словами `var`, `val`, `fun` в определении свойства или функции.

Если модификатор доступа явно не указан, то по умолчанию применяется модификатор `public`. Например:

```
class Person() {  
    var name = "Без имени" // Эквивалентно "public var name = "Без  
имени""  
  
    var age = 18           // Эквивалентно "public var age = 18"  
  
    fun displayPerson() {  
        println("Имя: $name Возраст: $age")  
    }  
}
```



ООП. Модификаторы видимости

Если свойства объявляются через первичный конструктор и для них не указан модификатор доступа, они также будут иметь модификатор `public`.

Например:

```
class Person(val name: String, val age: Int) {  
    fun displayPerson() {  
        println("Имя: $name  Возраст: $age")  
    }  
}
```



ООП. Модификаторы видимости

Модификатор `private` делает свойства и методы доступными только внутри класса, где они определены:

```
class Person(private val name: String, _age: Int) {  
    private val age = _age  
    fun displayPerson() {  
        displayName()  
        displayAge()  
    }  
    private fun displayName() {  
        println("Имя: $name")  
    }  
    private fun displayAge() {  
        println("Возраст: $age")  
    }  
}  
  
fun main() {  
    val andrew = Person("Andrew", 27)  
    andrew.displayPerson()  
  
    // println(andrew.name) // Ошибка! - свойство  
    // andrew.displayAge() // Ошибка! - функция  
    name - private  
    displayAge - private
```




ООП. Модификаторы видимости

Модификатор `protected` позволяет свойствам и методам быть доступными в классе, где они определены, а также в классах-наследниках:

```
open class Person(protected val name: String, private val age: Int) {
```

```
    protected fun displayPerson() {
```

```
        displayName()
```

```
        displayAge()
```

```
    }
```

```
    private fun displayName() {
```

```
        println("Имя: $name")
```

```
    }
```

```
    private fun displayAge() {
```

```
        println("Возраст: $age")
```

```
    }
```

```
}
```

Здесь свойство `name` и метод `displayPerson` доступны в классе `Employee`, который является наследником класса `Person`.

```
class Employee(name: String, age: Int) : Person(name, age) {
```

```
    fun displayEmployee() {
```

```
        println("Employee $name.")
```

```
        displayPerson() // Метод displayPerson доступен в наследнике
```

```
    }
```

```
}
```



ООП. Модификаторы конструкторов

Модификаторы конструкторов в Kotlin позволяют управлять доступом к конструкторам классов. Это относится как к первичным, так и к вторичным конструкторам. По умолчанию, если модификатор не указан явно, конструктор имеет модификатор доступа `public`, что означает, что он доступен из любой части программы.

```
class Person private constructor(val name: String) {  
    fun displayPerson() {  
        println("Имя: $name")  
    }  
}
```

Здесь первичный конструктор класса `Person` имеет модификатор `private`, что делает его недоступным для создания объектов класса извне этого класса.

Однако внутри самого класса `Person` можно использовать этот приватный конструктор в других конструкторах, что полезно для организации логики.



ООП. Геттеры и Сеттеры

Геттеры (getter) и сеттеры (setter) (методы доступа) позволяют управлять доступом к переменной.

Формальная запись

var имя_свойства[: тип_свойства] [= инициализатор_свойства]

[getter]

[setter]

Инициализатор, геттер и сеттер - это части свойства, которые можно определить, но не обязательно. Если не указывать тип свойства, компилятор Kotlin может автоматически вывести его из значения инициализатора или из типа, возвращаемого геттером.



ООП. Сеттер

Сеттер определяет логику установки значения переменной, который определяется с помощью слова `set`. Рассмотрим распространенный пример с возрастом. Есть переменная `age`, которая хранит возраст пользователя и представляет числовое значение.

```
var age: Int = 18
```

Установить можно любой возраст, однако отрицательный не является корректным, поэтому для проверки входных значений можно использовать сеттер.

```
var age: Int = 18
```

```
    set(value){
```

```
        if((value>0) and (value <110))
```

```
            field = value
```

```
    }
```

```
fun main() {
```

```
    println(age)  // 18
```

```
    age = 30
```

```
    println(age)  // 30
```

```
    age = -345
```

```
    println(age)  // 30
```

```
}
```



ООП. Сеттер

Блок "set" представляет собой часть кода, которая определяет процесс установки значения для свойства. Этот блок располагается сразу после определения свойства, к которому он относится. Фактически, блок "set" представляет собой функцию с одним параметром, называемым "value". Этот параметр "value" служит для передачи устанавливаемого значения в блок "set". Например, при выполнении операции "age = 30", число 30 передается как устанавливаемое значение и будет храниться в параметре "value".

Внутри блока "set" выполняется проверка, входит ли устанавливаемое значение в диапазон допустимых значений. Если устанавливаемое значение находится в этом диапазоне (т.е., оно корректно), то оно передается внутреннему полю "field" для хранения. В противном случае, если значение не соответствует допустимому диапазону, свойство остается с сохраненным предыдущим значением.

Идентификатор field представляет автоматически генерируемое поле, которое непосредственно хранит значение свойства



ООП. Геттер

Геттер управляет получением значения свойства и определяется с помощью ключевого слова `get`:

```
var age: Int = 18  
    set(value){  
        if((value>0) and (value <110))  
            field = value  
    }  
    get() = field
```

.



ООП. Геттер

После выражения "get()" через знак "=" указывается, что будет возвращено при обращении к этому свойству. В данном случае, значение поля "field", которое содержит информацию о свойстве "name". Однако, в данном геттере нет большой необходимости, так как значение "name" можно получить и без использования геттера.

Если геттер должен содержать больше инструкций, то геттер можно оформить в блок с кодом внутри фигурных скобок:

```
var age: Int = 18
```

```
    set(value){
```

```
        println("Call setter")
```

```
        if((value>0) and (value <110))
```

```
            field = value
```

```
    }
```

```
    get(){
```

```
        println("Call getter")
```

```
        return field
```

```
    }
```

Если геттер оформлен в блок кода, то для возвращения значения необходимо использовать оператор `return`



ООП. Инкапсуляция

Инкапсуляция - это один из принципов объектно-ориентированного программирования, который предполагает сокрытие деталей реализации объекта от внешнего мира и предоставление доступа к объекту только через определенные интерфейсы (методы, свойства). В Kotlin инкапсуляция реализуется с помощью модификаторов доступа и свойств (properties).



ООП. Инкапсуляция

Например, рассмотрим пример с классом Student, который демонстрирует инкапсуляцию в Kotlin

```
class Student(private val name: String, private var age: Int) {
```

```
    fun haveBirthday() {
```

```
        age++
```

```
        println("$name $age лет.")
```

```
    }
```

```
    fun getName(): String {
```

```
        return name
```

```
    }
```

```
    fun getAge(): Int {
```

```
        return age
```

```
    }
```

```
}
```



ООП. Инкапсуляция

В примере создается класс `Student`, который имеет два приватных свойства: `name` (имя студента) и `age` (возраст студента). Эти свойства недоступны снаружи класса.

Метод `haveBirthday` увеличивает возраст студента на 1 и выводит сообщение об изменении возраста.

Свойства `name` и `age` инкапсулируют доступ к данным студента, предоставляя методы `getName` и `getAge` для получения имени и возраста студента соответственно.

```
fun main() {  
    val student = Student("Алиса", 20)  
    student.haveBirthday()  
    println("${student.getName()} ${student.getAge()} лет.")  
}
```



Спасибо за внимание!